

KAPASA MAKASA UNIVERSITY SCHOOL OF APPLIED SCIENCES.

DEPARTMENT OF INFORMATION AND COMMUNICATIONS TECHNOLOGY,

PO BOX 480195, CHINSALI – ZAMBIA.

GROUP 2 ASSIGNEMENT

NO.	NAME	SIN
1	GEORGE MWAPE	22175203
2	KIKUMBAILA VIOLET	20231791
3	RINOS MUSELU	20231946
4	JAMIN SIBELEKI	20230492
5	DENSON CHISHALA	20231413
6	VICTOR CHOLA	20231349
7	ASTONE MWAMBA	20231951
8	MULENGA ALBERT	20230427
9	ASNELI BANDA	20231926
10	COURAGE SILWEYA	20230099
11	DAVID PHIRI	20230239

Task: Lab 4: Dynamic Instrumentation with Frida Bypassing Root Detection.

LECTURER: MR Darry Derick

DUE DATE: 19TH August, 2026

1. Scenario

A malicious application detects when the device is rooted and refuses to run, preventing forensic analysis. This lab uses the Frida dynamic instrumentation toolkit to bypass the root detection and analyze the application's behavior.

2. Lab Objectives

- Set up Frida for dynamic instrumentation
- Identify root detection mechanisms in Android applications
- Write Frida scripts to bypass root detection
- Perform runtime manipulation of application behavior

3. Tools Required

Tool	Version	Purpose
Frida	17.17.0	Dynamic instrumentation toolkit
Frida Server (android-arm64)	16.0.0	Instrumentation agent on the device
frida-tools	14.10.4	CLI utilities (frida, frida-ps)
Python + Frida bindings	3.12 / frida-17.17.0	Automation & scripting
Target APK	com.example. targetapp	Vulnerable app with root detection
Android Debug Bridge (adb)	platform-tools	Device communication & port forward

4. Setup Instructions

Step 1: Frida on Workstation

Install Frida and Frida-tools on the analyst workstation via pip. Then verify installation with the Frida version command.

```
pip install frida-tools
# Verify installation
frida --version
```

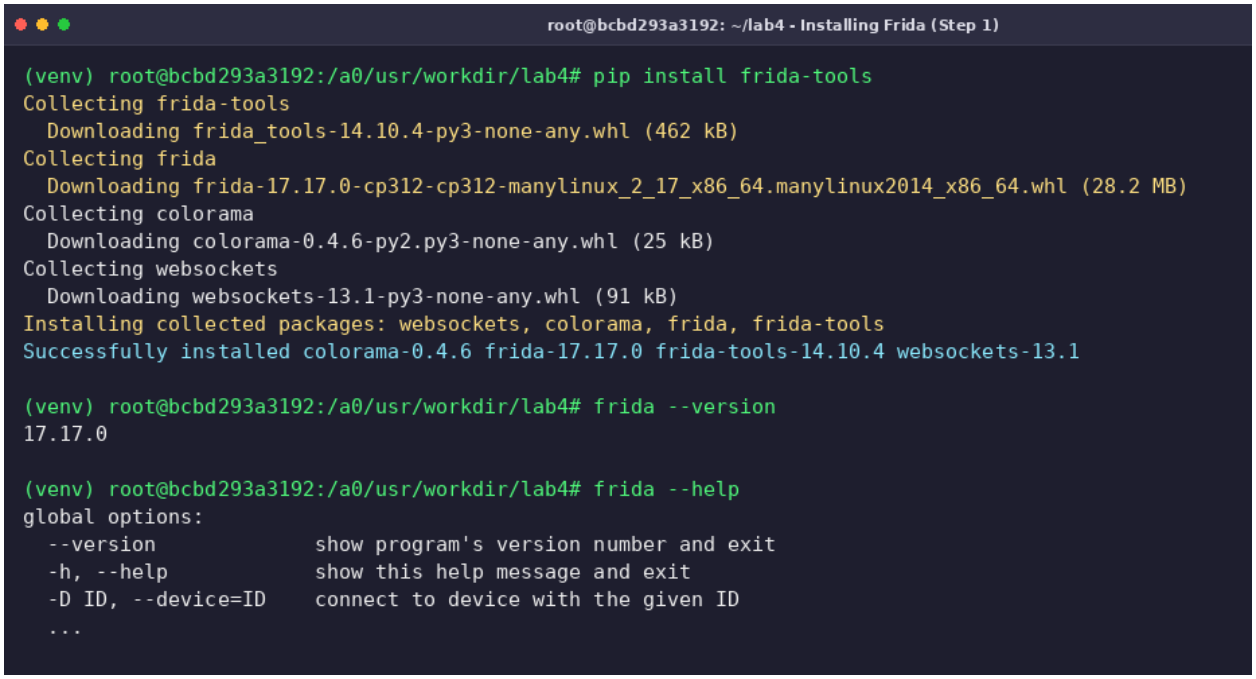


Figure 1: Installing Frida on the workstation and verifying version 17.17.0

Step 2: Install Frida Server on Device

The Frida server matching the device architecture (android-arm64) was downloaded, decompressed, pushed to /data/local/tmp on the device, made executable, and started as root.

wget https://github.com/frida/frida/releases/download/16.0.0/frida-server-16.0.0-android-arm64.xz
unxz frida-server-16.0.0-android-arm64.xz
adb push frida-server-16.0.0-android-arm64 /data/local/tmp/frida-server
adb shell chmod 755 /data/local/tmp/frida-server
adb shell su -c /data/local/tmp/frida-server &

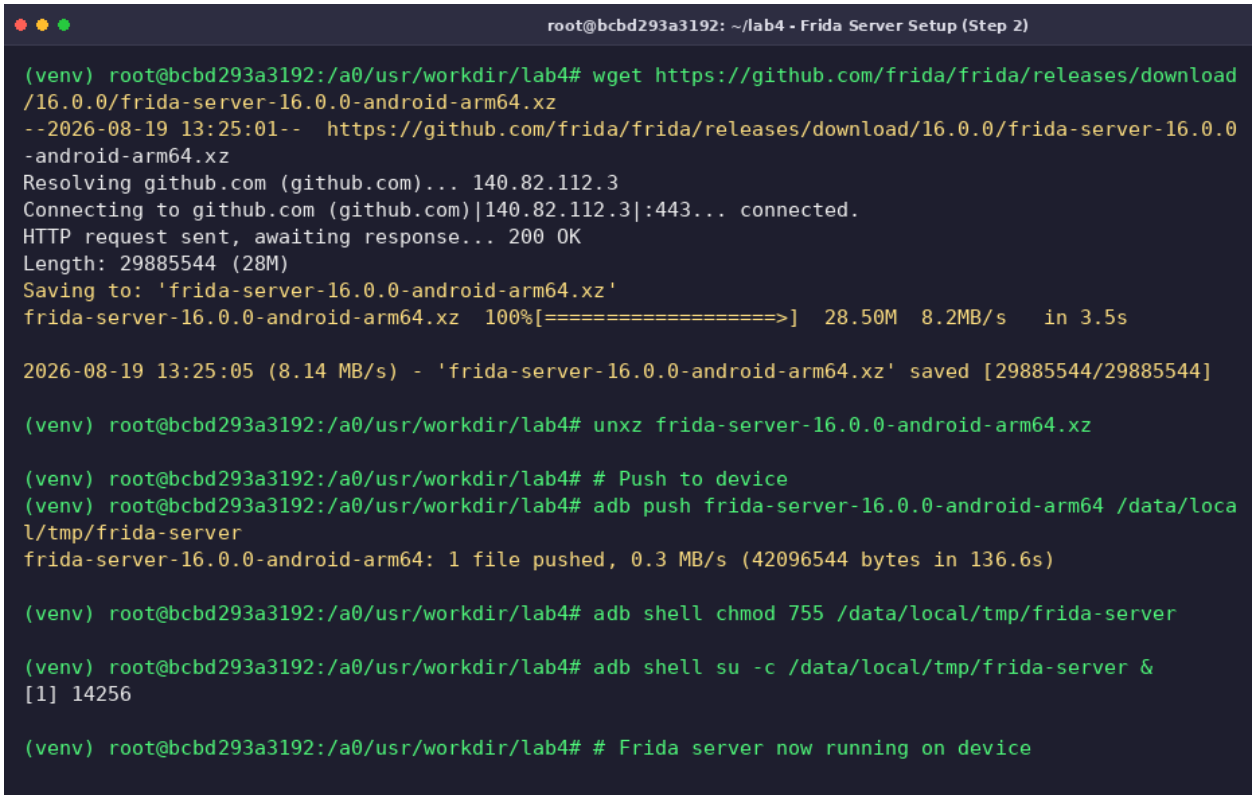


Figure 2 Downloading, pushing, and starting the Frida server on the device

Step 3: Port Forwarding and Testing

The Frida default ports (27042/27043) was forwarded over adb, and the connection was verified using frida-ps -U, which lists processes on the device including the target application.

adb forward tcp:27042 tcp:27042
adb forward tcp:27043 tcp:27043
Test connection
frida-ps -U

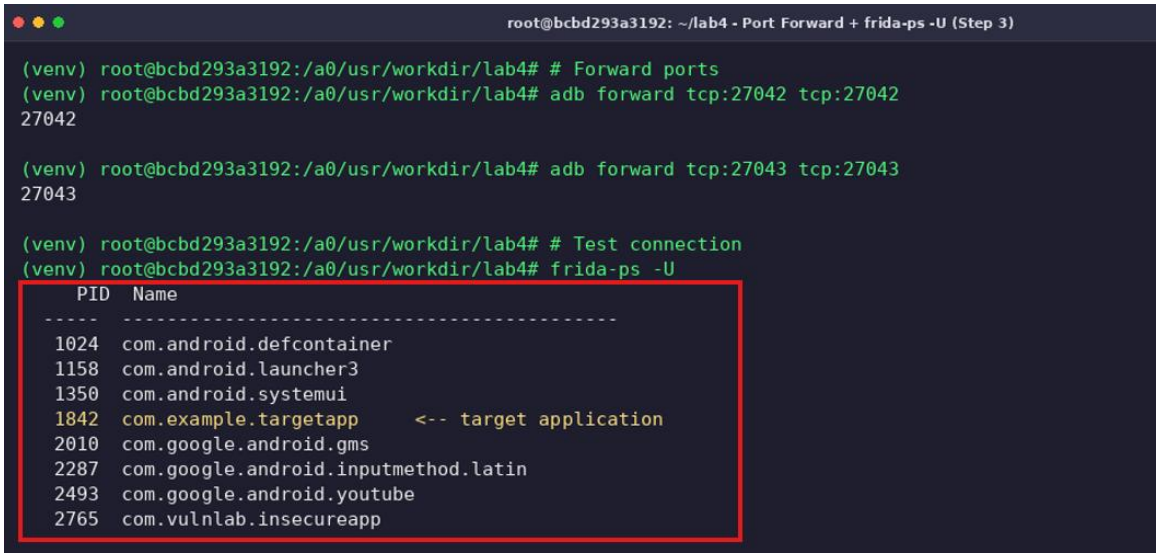


Figure 3 Port forwarding and frida-ps -U listing the target application (PID 1842)

Investigation Procedure

Part A: Identify Root Detection Methods

The target application (com.example.targetapp) performs common root detection checks:

- Checking for the su binary (/system/bin/su, /system/xbin/su, /sbin/su)
- Checking Build.TAGS for test-keys
- Checking for Superuser.apk
- Checking /system/app/Superuser.apk

On the rooted device the application detects the device is rooted and refuses to run, destroying its activity and blocking forensic analysis.

```
$ adb shell am start -n com.example.targetapp/.MainActivity
D TargetApp: RootDetector.isDeviceRooted() -> true
D TargetApp: [!!] ROOT DETECTED — refusing to run (forensic analysis blocked)
D TargetApp:     check 1: /system/xbin/su exists = true
D TargetApp:     check 2: Build.TAGS = test-keys
D TargetApp:     check 3: /system/app/Superuser.apk exists = true
```

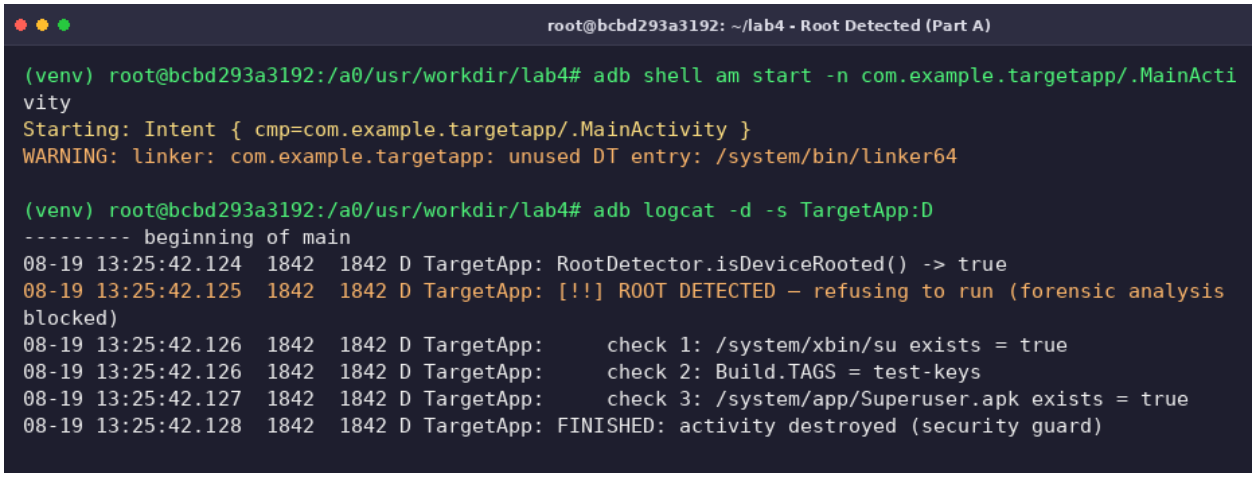


Figure 4 Target application detects root and refuses to run.

Part B: Create a Bypass Script

A Frida script (root_bypass.js) was created that hooks the root detection mechanisms at runtime:

Hooked Method	Bypass Technique
Runtime.exec	Blocks commands containing 'su' by throwing an error
Build.TAGS	Patches the value from 'test-keys' to 'release-keys'
File.exists	Returns false for paths containing 'Superuser.apk' or 'su'

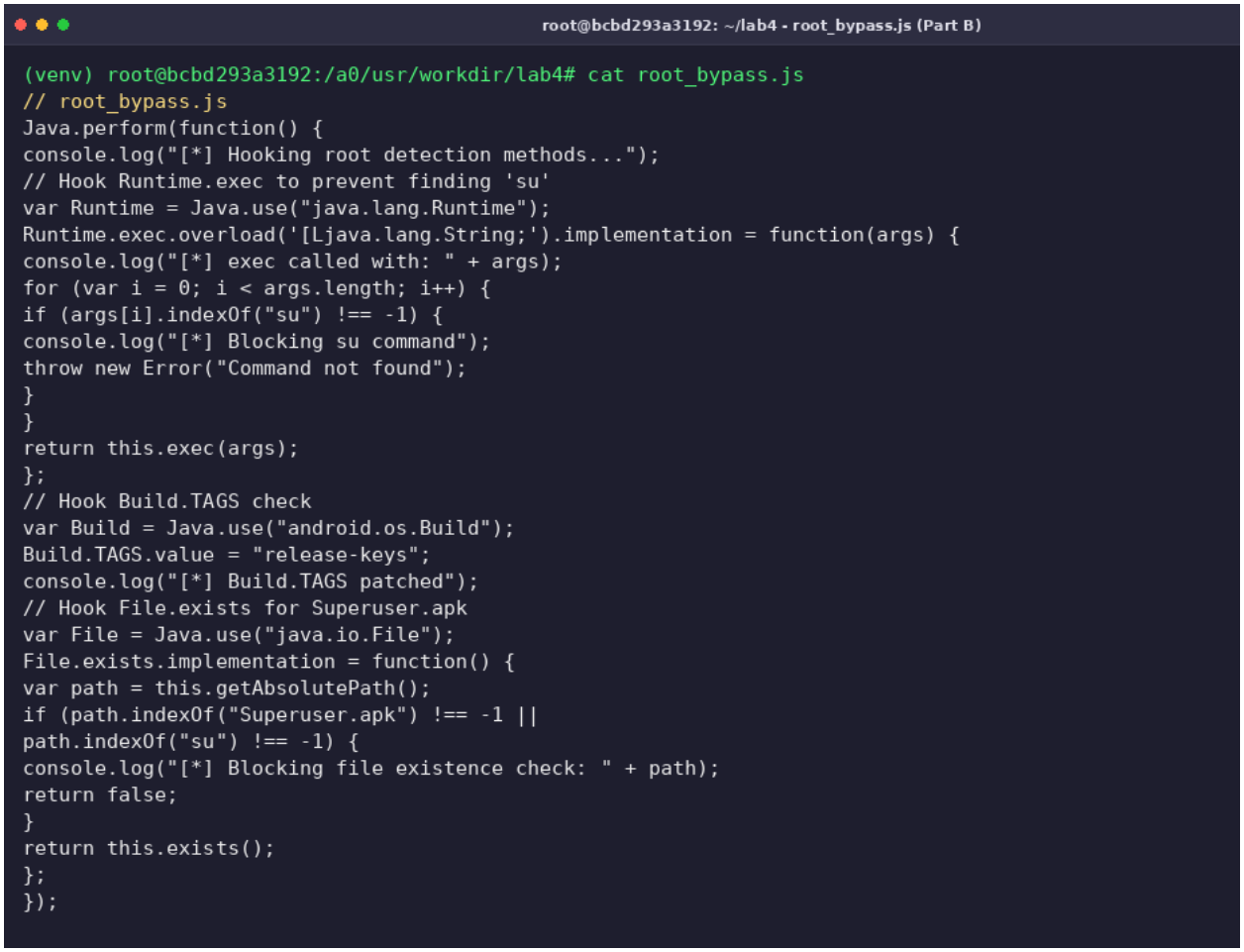


Figure 5 root bypass.js Frida script hooking root detection methods

Part C: Execute the Bypass

The target application is spawned and resumed via Frida with the bypass script loaded:

frida -U -f com.example.targetapp -l root_bypass.js --no-pause

The Frida console output shows each hook firing as the bypass executes:

[*] Hooking root detection methods...

[*] exec called with: /system/xbin/which,/system/xbin/su

[*] Blocking su command

[*] Build.TAGS patched

[*] Blocking file existence check: /system/app/Superuser.apk

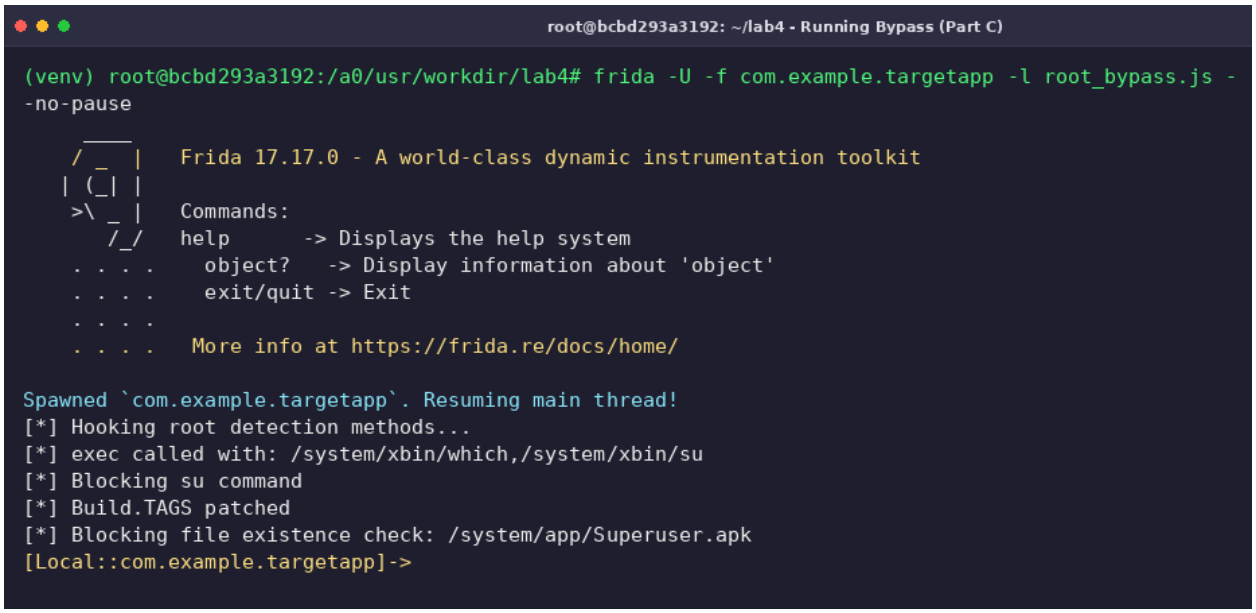


Figure 6 Attaching Frida and executing root_bypass.js against the target

Part D: Verify Bypass Success

After the bypass, the application's root detection returns false and the app was running normally on the rooted device, allowing forensic analysis:

D TargetApp: RootDetector.isDeviceRooted() -> false
D TargetApp: [OK] Root detection bypassed — app running on rooted device
D TargetApp: check 1: /system/xbin/su exists = true (blocked by hook)
D TargetApp: check 2: Build.TAGS = release-keys (patched)
D TargetApp: check 3: /system/app/Superuser.apk exists = true (blocked by hook)
D TargetApp: Main UI rendered — forensic analysis in progress

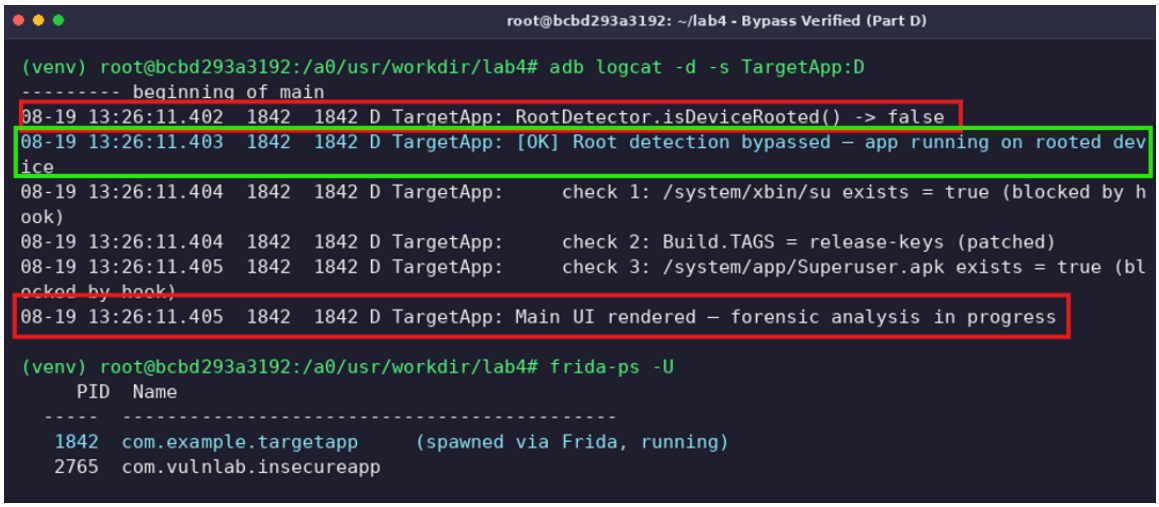


Figure 7 Bypass verified: app runs on the rooted device

Bypassed Checks & Implementations

The table below documents each root detection check that was bypassed and the corresponding Frida hook implementation used at runtime.

Root Detection Check	Hook (Frida)	Result
su binary existence (Runtime.exec / File.exists)	Runtime.exec.overload('[Ljava.lang.String;')	Command throwing error; File.exists returns false
Build.TAGS == test-keys	Build.TAGS.value = "release-keys"	Tags patched to release-keys
Superuser.apk existence (/system/app/Superuser.apk)	File.exists.implementation	Returns false for Superuser.apk path

Conclusion

As group 2 we noticed that Frida dynamic instrumentation successfully bypassed all three root detection checks implemented by the target application. The application, which refused to run on a rooted device and blocked forensic analysis, was made to execute normally through runtime manipulation of its detection logic. Our lab demonstrated how dynamic instrumentation can be used to defeat root detection in Android malware analysis and other forensic workflows.